

程序设计Ⅶ

张宏斌 副教授

复习

- Python序列：有序 vs 无序；可变 vs 不可变
- 序列的操作：索引与切片；+，*，in，not in，count()
- 列表
 - 创建与删除
 - 常用方法
 - 添加：append(), insert(), extend()
 - 删除：pop(), remove(), del命令
 - 寻找：index(), count()
 - 排序：sort(), reverse()
 - 内置函数：sorted(), max(), min(), sum(), len(),
 - 列表推导式
 - 遍历列表
- 元组：元组定义；序列解包；元组的用途

第五章 字典与集合

- 1 掌握字典类型的基本概念和操作
- 2 熟悉字典类型的常用方法
- 3 掌握集合类型的基本概念和操作
- 4 熟悉集合类型的常用方法
- 5 灵活运用字典和集合类型解决实际问题

字典

- 字典 (dictionary) 是包含若干“键:值”元素的**无序可变序列**，字典中的**每个元素包含用冒号分隔开的“键”和“值”两部分**，表示一种映射或对应关系。定义字典时，每个元素的“键”和“值”之间用冒号分隔，不同元素之间用逗号分隔，所有的元素放在一对**大括号“ {} ”**中
- 字典中元素的“键”可以是**Python中任意不可变数据**，例如**整数、实数、复数、字符串、元组**等可哈希数据类型，但不能使用列表、集合、字典或其他可变类型作为字典的“键”
- 字典中的“键”不允许重复，“值”是可以重复的

```
{<key1>: <value1>, <key2>: <value2>, ..., <keyN>: <valueN>}
```

```
stock_dic = {'600000':11.54, '600036':36.84, '600048':16.61, '000001':14.73}  
print(stock_dic)
```

创建字典

- 创建字典，可以通过一对花括号“{}”或者 **dict()** 函数来创建

```
stock_dic = {'600000': '浦发银行', '600036': '招商银行', '600048': '保利地产', '000001': '平安银行'}  
print(stock_dic)  
empty_dic = {}  
print(empty_dic)
```

```
{'600000': '浦发银行', '600036': '招商银行', '600048': '保利地产', '000001': '平安银行'}  
{}
```

- 使用 **dict()** 函数创建字典时，可以将由键值元组构成的序列作为参数

```
stock_dic = dict([('600000', '浦发银行'), ('600036', '招商银行'),  
                  ('600048', '保利地产'), ('000001', '平安银行')])  
print(stock_dic)
```

```
{'600000': '浦发银行', '600036': '招商银行', '600048': '保利地产', '000001': '平安银行'}
```



字典元素的访问

- 字典中的每个元素表示一种映射关系或对应关系，根据提供的“**键**”作为**下标**就可以访问对应的“**值**”，如果字典中不存在这个“**键**”会抛出异常

```
>>> aDict = {'age': 39, 'score': [98, 97], 'name': 'Dong', 'sex': 'male'}  
>>> aDict['age']           # 指定的“键”存在，返回对应的“值”  
39  
>>> aDict['address']      # 指定的“键”不存在，抛出异常  
KeyError: 'address'
```



修改字典的元素或添加元素

- 当以指定“键”为下标为字典元素**赋值**时，有**两种含义**：
- **修改字典的值**，当键存在时，可以通过类似列表元素赋值的方式，给字典的键赋值来修改键所对应的值

```
>>> stock_dic = {'600000': '浦发银行', '600036': '招商银行', '600048': '保利地产', '000001': '平安银行'}
>>> stock_dic['600000'] = '上海浦发银行'
>>> stock_dic
{'600000': '上海浦发银行', '600036': '招商银行', '600048': '保利地产', '000001': '平安银行'}
```

- **添加键值对**，当键不存在时，也可以为它赋值，这样Python会自动为该字典添加新的键值对

```
>>> stock_dic['000063'] = '中兴通讯'
>>> stock_dic
{'600000': '上海浦发银行', '600036': '招商银行', '600048': '保利地产', '000001': '平安银行', '000063': '中兴通讯'}
```



字典元素的访问（续）

- 字典对象提供了一个**get()**方法用来返回指定“键”对应的“值”，并且允许指定该键不存在时返回特定的“值”（默认为空值None）。**get()不会改变字典**，例如：

```
>>> aDict.get('age')           # 如果字典中存在该“键”则返回对应的“值”
39
>>> aDict.get('address', 'Not Exists.') # 指定的“键”不存在时返回指定的默认值
'Not Exists.'
```

- 而另一个类似的方法**setdefault()**则会**改变字典**。如果该键确实存在，方法就会返回键的值，不然在字典中添加数据

```
>>> aDict.setdefault('address', 'Zhongda.')
```



字典的常用方法

- 字典有些操作和序列类型相似。例如，可以通过 `len()` 函数获取字典的长度，即键值对的个数，也可以通过 `in` 来判断键是否在字典当中，还可以通过 `copy()` 方法进行复制。除此之外，字典还有多种特定的方法，如表所示

字典的常用方法

方法	描述
<code>.keys()</code>	返回包含所有键的可迭代对象
<code>.values()</code>	返回包含所有值的可迭代对象
<code>.items()</code>	返回包含所有键值对构成元组的可迭代对象
<code>.get()</code>	键存在则返回对应值，否则返回默认值
<code>.setdefault()</code>	键存在则返回对应值，否则添加键值对，并返回值
<code>.pop()</code>	键存在则返回对应值，并删除键值对，否则返回默认值
<code>.popitem()</code>	以元组 (key, value) 形式随机返回并删除键值对
<code>.clear()</code>	删除所有键值对



字典元素的访问（续）

- 使用字典对象的 `items()` 方法可以返回字典的键、值对：
`aDict.items()`
- 使用字典对象的 `keys()` 方法可以返回字典的键
- 使用字典对象的 `values()` 方法可以返回字典的值
- 可利用方法 `items()`，编写用于遍历字典的 `for` 循环，可声明两个变量，用于存储键-值对里面的键、值

for key, value in aDict.items():

 print("\nKey", key)

 print("\nValue", value)

- 可以遍历所有的键或者值：

for key in aDict.keys():

 process

for value in aDict.values():

 process



字典元素的添加、修改与删除

- 使用字典对象的**update()**方法可以将另一个字典的“键:值”一次性全部**添加到当前字典对象**，如果两个字典中存在**相同的“键”**，则以**另一个字典中的“值”为准**对当前字典进行更新

```
>>> aDict = {'age': 37, 'score': [98, 97], 'name': 'Dong', 'sex': 'male'}
>>> aDict.update({'a': 97, 'age': 39}) # 修改'age'键的值，同时添加新元素'a':97
>>> aDict
{'age': 39, 'score': [98, 97], 'name': 'Dong', 'sex': 'male', 'a': 97}
```

字典元素的添加、修改与删除

- 如果需要删除字典中指定的元素，可以使用**del**命令

```
>>> del aDict['age'] # 删除字典元素
>>> aDict
```

```
{'score': [98, 97], 'name': 'Dong', 'sex': 'male', 'a': 97}
```

- 也可以使用字典对象的**pop()**和**popitem()**方法弹出并删除指定的元素，**pop()**应指定拟删除的键，**popitem()**则不能指定，例如：

```
>>> aDict = {'age': 37, 'score': [98, 97], 'name': 'Dong', 'sex': 'male'}
>>> aDict.popitem() # 弹出一个元素，对空字典会抛出异常
```

```
('sex', 'male')
```

```
>>> aDict.pop('age') # 弹出指定键对应的值
37
```

```
>>> aDict
```

```
{'score': [98, 97], 'name': 'Dong'}
```

- **clear()** 方法用于清除字典中所有的键值对

计数神器Counter()

- **Counter()** 将一个序列的值转化为一个类似于整型的标准字典的键到计数的对象映射。我们主要用它来生成计数器，注意参数为**列表或其他可迭代对象**：

```
from collections import Counter
```

```
c = Counter([0, 1, 2, 0])    # c是 {0: 2, 1: 1, 2: 1}
```

- 这给我们提供了一个用来解决单词计数问题的很简便的方法：

```
word_counts = Counter(words_list)
```

```
from collections import Counter
```

```
words = ['look', 'into', 'my', 'eyes', 'look', 'into', 'my', 'eyes',  
'the', 'eyes', 'the', 'eyes', 'the', 'eyes', 'not', 'around', 'the',  
'eyes', "don't", 'look', 'around', 'the', 'eyes', 'look', 'into',  
'my', 'eyes', "you're", 'under']
```

```
words_count = Counter(words)
```

对单词计数且存在一个字

典中

```
print(words_count)
```

```
top_three = words_count.most_common(3)
```

寻找频数最高的三个单

词

```
print(top_three)
```

```
morewords = ['why', 'are', 'you', 'not', 'looking', 'in', 'my', 'eyes']
words_count.update(morewords)           # 将morewords中的单词计数追加到words_count中
print(words_count)
top_three = words_count.most_common(3)
print(top_three)

a = Counter(words)
b = Counter(morewords)
c = a + b                                # 两个计数的合计
print(c)
d = a - b                                # 两个计数的差
print(d)
```



字典推导式

- 字典推导式利用每次迭代收集表达式的键和值结果，并将该键值对添加到新的字典中。语法形式如下：

```
{key_expression: value_expression for expression in iterable}
```

```
>>> stock_code = ['600000', '600036', '600048', '000001']
>>> close_price = [11.54, 36.84, 16.61, 14.49]
>>> stock_dic = dict(zip(stock_code, close_price))
>>> print(stock_dic)
{'600000': 11.54, '600036': 36.84, '600048': 16.61, '000001': 14.49}
>>> stock_dic = {code: price for code, price in zip(stock_code, close_price)}
>>> print(stock_dic)
{'600000': 11.54, '600036': 36.84, '600048': 16.61, '000001': 14.49}
```



字典推导式（续）

- 字典推导式也可以加上 if 条件语句进行数据的过滤。语法形式如下：

```
{key_expression: value_expression for expression in iterable if condition}
```

例如，生成仅包含沪市主板 A 股股票数据的字典，即股票代码以“60” 开头

```
>>> stock_dic = {code: price for code, price in zip(stock_code, close_price)
                 if code.startswith('60')}
>>> stock_dic
{'600000': 11.54, '600036': 36.84, '600048': 16.61}
```

列表作业问题

集合

- **集合**类型与数学中集合的概念是一致的。它是由 0 个或多个**唯一的、不可变**的元素构成的无序组合
- **集合**也是通过一对花括号“{}”来标识的。不同的是，集合中的元素不是键值对。我们可以把集合看作是去掉了值，仅仅保留键的字典
- **集合**中的**元素是不重复的**，常常用于去重或检查某个元素是否存在
- 集合中的**元素**类型只能是整型、浮点型、字符串和元组等**不可变类型**
- **集合本身是可变的**，可以添加、删除和清空集合元素



创建集合

- 可以使用 **set()** 函数创建一个集合，或者用花括号将一系列以逗号隔开的元素包裹起来创建。使用 **set()** 函数时，传入一个可迭代对象作为参数即可

```
>>> print(set(['浦发银行', '招商银行', '农业银行', '建设银行']))
>>> print({'600000', '600036', '601288', '601939'})
>>> print(set())          # set() 创建空集合
print(type({}))

{'浦发银行', '建设银行', '农业银行', '招商银行'}
{'601939', '600000', '601288', '600036'}
set()
<class 'dict'>
```

- 需要注意的是，字典和集合都使用花括号进行标识。但使用“{}”创建的是**空字典**，而不是空集合。创建空集合应该使用 **set()** **函数**



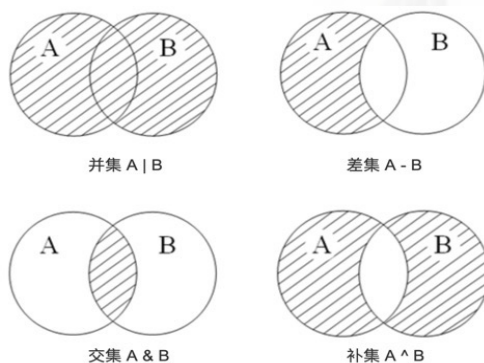
集合的元素不重复

- 集合的重要特征之一就是**元素不重复**。因此，当使用 `set()` 函数将其他可迭代对象转换为集合时，该函数会自动过滤掉重复元素

```
>>> print(set('好好学习, 天天向上'))  
{',', '天', '学', '好', '习', '上', '向'}  
  
>>> print({'浦发银行', '招商银行', '农业银行', '建设银行', '招商银行', '农业银行'})  
{',', '浦发银行', '建设银行', '农业银行', '招商银行'}
```

集合的运算

- 集合还有四个常用的操作：**交集 (&)**、**并集 (|)**、**差集 (-)**、**补集 (异或集) (^)**，操作逻辑与数学定义相同。A、B 两个集合的运算示意图如图所示，阴影部分是运算结果



集合的运算（举例）

```
>>> stock_name_set1 = {'建设银行', '浦发银行', '招商银行', '农业银行'}
stock_name_set2 = {'工商银行', '中国银行', '招商银行', '农业银行'}

>>> stock_name_set1 | stock_name_set2
{'中国银行', '农业银行', '工商银行', '建设银行', '招商银行', '浦发银行'}

>>> stock_name_set1 - stock_name_set2
{'建设银行', '浦发银行'}

>>> stock_name_set1 & stock_name_set2
{'农业银行', '招商银行'}

>>> stock_name_set1 ^ stock_name_set2
{'中国银行', '工商银行', '建设银行', '浦发银行'}
```



集合的常用方法

- 集合可以通过内置函数 **len()** 得到元素个数，通过 **in** 检查某个元素是否存在，通过 **copy()** 方法复制集合。除此之外，还有一些特有的方法，如表所示

集合的常用方法

方法	描述
.add(x)	如果 x 不存在，将 x 添加到集合中
.clear()	移除集合中的所有元素
.pop()	随机返回并删除集合中的一个元素
.discard(x)	移除集合中的 x；如果 x 不存在，不报错
.remove(x)	移除集合中的 x；如果 x 不存在，触发 KeyError 异常



add()方法

- **add()** 方法，与列表、字典一样，集合是可变类型。可以往集合中添加元素。通过集合的 **add()** 方法来实现。如果元素本身在集合中，则不添加

```
>>> bank_name = {'建设银行', '浦发银行', '招商银行', '农业银行'}
>>> bank_name.add('工商银行')
>>> bank_name
{'农业银行', '工商银行', '建设银行', '招商银行', '浦发银行'}
```



删除元素

- 移除元素的**pop()**、**discard()**、**remove()**和**clear()**方法。其中，**pop()**方法会随机移除并返回一个元素；**discard()**和**remove()**方法可以将指定的元素从集合中移除，区别在于当需要移除的元素不在集合中时，**discard()**方法不报错，而**remove()**方法会触发**KeyError**错误；**clear()**方法会移除集合中的所有元素

```
>>> bank_name = {'建设银行', '浦发银行', '招商银行', '农业银行'}
>>> bank_name.pop()
'浦发银行'
>>> bank_name
{'农业银行', '建设银行', '招商银行'}
>>> bank_name.discard('中国银行')
```

```
>>> bank_name.remove('中国银行')
Traceback (most recent call last):
  File "<pyshell#10>", line 1, in <module>
    bank_name.remove('中国银行')
KeyError: '中国银行'
>>> bank_name.remove('招商银行')
bank_name
{'农业银行', '建设银行'}
>>> bank_name.clear()
bank_name
set()
```



集合推导式

- **集合推导式**使用花括号“{}”生成集合。集合推导式在 for 关键字前的表达式结果是一个元素，而不是字典推导式的键值对。完整的语法形式如下

```
{expression for item in iterable if condition}
```

将可迭代对象 iterable 中的每个元素赋值给 item 后，当条件表达式 condition 为 True 时，利用表达式 expression 进行处理，得到的结果构成新集合的元素

```
>>> stock_code = ['600000', '600036', '600048', '000001']
>>> sh_stock_code = {'sh_' + item for item in stock_code if item.startswith('60')}
>>> sh_stock_code
{'sh_600048', 'sh_600036', 'sh_600000'}
```



总结

- Python数据类型总结

组合数据类型比较

类型	标识符	可变	有序	元素
字符串	"或"	✗	✓	字符
列表	[]	✓	✓	任意类型
元组	()	✗	✓	任意类型
字典	{}	✓	✗	键值对，键为不可变类型且不重复
集合	{}	✓	✗	任意不可变类型，不重复

- 字典定义
- 字典元素的访问、修改、添加——通过字典的键进行



总结

- 字典元素的访问：get()、setdefault()方法
- 字典元素的访问：items()、keys()、values()方法
- 遍历字典元素
- 字典元素的添加、修改与删除
 - del命令
 - update()、pop()、popitem()、clear()方法
- 计数神器Counter()函数
- 字典推导式



总结

- 集合的定义
- 集合的运算：| & - ^
- 集合的常用方法

集合的常用方法

方法	描述
.add(x)	如果 x 不存在，将 x 添加到集合中
.clear()	移除集合中的所有元素
.pop()	随机返回并删除集合中的一个元素
.discard(x)	移除集合中的 x；如果 x 不存在，不报错
.remove(x)	移除集合中的 x；如果 x 不存在，触发 KeyError 异常



作业

- 写一个程序，能够对学院网站中的英文介绍的文本内容（见课件网 **About_Lingnan.txt**），进行单词计数（注意要去掉标点符号），并输出频数最高的10个单词
- 作业务必按照模版 **hw03_template.py** 的格式写，模版在课件网上下载，建议直接在模版上完成作业，**务必记得修改文件名称**
- **注意输出格式有特殊要求！！！！**
- 作业要求是.py文件，务必命名为：**hw03_学号.py**
- 作业提交网址和上次相同，截止时间为下周二晚上11点50

作业（自愿作业）

- 做一个Excel表，总结字典和集合
 - 总结字典和集合的定义方法、常用操作等方法
 - 可举例